

Laporan Praktikum Minggu Ke-5 Kontrol Cerdas

Minggu ke-5

Nama : Agustinus Caecar Bhakti Abiyoga

NIM : 224308051

Kelas : TKA 7C

Akun Github (Tautan) : [AgustinusCBA224308051TKA7C \(Yoga Agustinus \)](#)

1. Judul Percobaan

Deep Reinforcement Learning untuk Kontrol Kompleks

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.
2. Memahami konsep dasar kontrol cerdas (intelligent control systems). Mengimplementasikan Deep Q-Network (DQN) untuk kontrol otomatis.
3. Memahami konsep Deep Reinforcement Learning (DRL) dalam kontrol sistem kompleks

3. Landasan Teori

Deep Reinforcement Learning (DRL) adalah pengembangan dari Reinforcement Learning (RL) yang memanfaatkan jaringan saraf (Neural Network) untuk menangani sistem dengan kondisi yang kompleks dan bersifat kontinu. Dalam RL, agen belajar dengan berinteraksi langsung dengan lingkungan guna memperoleh reward maksimal, sedangkan DRL menggabungkan kemampuan Deep Learning yang mampu mengenali pola dan memperkirakan nilai tindakan (Q-value) secara lebih efektif (Sutton & Barto, 2018).

Salah satu algoritma kunci dalam DRL adalah Deep Q-Network (DQN), yang menggantikan tabel Q tradisional dengan jaringan saraf yang memperkirakan nilai Q untuk setiap aksi berdasarkan kondisi lingkungan saat itu. Algoritma ini menerapkan experience replay untuk menjaga kestabilan proses pelatihan dan menggunakan strategi epsilon-greedy untuk mengatur keseimbangan antara eksplorasi dan eksploitasi (Mnih et al., 2015). DQN sering diterapkan pada sistem kontrol yang kompleks, seperti robot otonom, kendaraan tanpa

pengemudi, dan optimasi penggunaan energi, karena mampu beradaptasi dengan perubahan lingkungan tanpa memerlukan model matematis sistem yang rinci.

4. Diskusi

1. Kelebihan dan Kelemahan DQN dibandingkan Metode Q-Learning Klasik

Deep Q-Network (DQN) menawarkan beberapa keunggulan dibandingkan metode Q-Learning tradisional, terutama dalam hal kapasitasnya untuk menangani sistem yang kompleks dengan ruang keadaan yang sangat besar. Pada Q-Learning klasik, pembelajaran dilakukan melalui tabel Q (Q-Table) yang menyimpan nilai untuk setiap kombinasi keadaan dan aksi. Metode ini berjalan efektif pada sistem yang sederhana dengan jumlah keadaan yang terbatas, namun menjadi tidak praktis ketika ruang keadaannya sangat luas atau kontinu. DQN mengatasi masalah ini dengan memanfaatkan jaringan saraf tiruan (Neural Network) untuk memperkirakan nilai Q secara langsung, sehingga lebih mampu berfungsi pada lingkungan yang lebih kompleks dan berubah-ubah. Selain itu, DQN memiliki fitur *experience replay* yang memungkinkan pengambilan sampel pengalaman secara acak selama proses pelatihan, sehingga dapat mengurangi korelasi antar data dan memperkuat stabilitas pembelajaran. Penggunaan strategi *epsilon-greedy* dalam DQN juga membantu menjaga keseimbangan antara eksplorasi dan eksploitasi, sehingga agen dapat belajar dengan lebih optimal dari lingkungannya.

Di sisi lain, DQN memiliki beberapa kelemahan, seperti kebutuhan sumber daya komputasi yang tinggi akibat proses pelatihan jaringan saraf yang kompleks, serta waktu pelatihan yang relatif lebih lama dibandingkan dengan Q-Learning konvensional. Selain itu, kinerja DQN sangat bergantung pada pemilihan parameter seperti *learning rate*, *discount factor*, dan ukuran jaringan, yang jika tidak disetel dengan baik dapat menyebabkan model mengalami *overfitting* atau ketidakstabilan dalam proses pembelajaran. Oleh karena itu, meskipun DQN menawarkan kemampuan yang lebih maju, penerapannya membutuhkan keahlian teknis dan perancangan yang teliti.

2 Adaptasi DQN untuk Kontrol Sistem Dinamis seperti Drone atau Robot Industri

Untuk mengaplikasikan DQN pada kontrol sistem dinamis seperti drone atau robot industri, diperlukan beberapa penyesuaian agar algoritma dapat beroperasi secara efektif di lingkungan yang terus berubah. Langkah pertama adalah menentukan state dan action secara tepat. Pada sistem drone misalnya, state dapat meliputi posisi, orientasi, dan kecepatan, sementara action bisa berupa perubahan sudut baling-baling atau gaya dorong. Data dari sensor seperti IMU, kamera, atau encoder digunakan sebagai input agar agen DQN memperoleh informasi yang akurat mengenai kondisi sistem secara real time.

Pelatihan DQN untuk sistem dinamis umumnya dimulai dalam lingkungan simulasi seperti OpenAI Gym, Gazebo, atau PyBullet untuk menghindari risiko kerusakan perangkat fisik selama tahap awal pembelajaran. Setelah model menunjukkan performa yang stabil dalam simulasi, hasil pelatihan tersebut dapat diterapkan ke perangkat nyata melalui teknik transfer learning. Selain itu, dalam implementasi dunia nyata, DQN sering dikombinasikan dengan metode kontrol konvensional seperti PID atau Fuzzy Logic guna menjaga kestabilan sistem saat model masih berproses beradaptasi.

Agar proses pembelajaran berjalan efektif, reward function perlu dirancang dengan seksama sesuai tujuan kontrol, misalnya menjaga keseimbangan drone atau mengarahkan robot ke posisi target dengan efisien. Dengan desain yang tepat, DQN mampu beradaptasi secara mandiri terhadap perubahan lingkungan dan memberikan kontrol yang akurat pada sistem dinamis yang kompleks

5. Assignment

Pada tugas minggu kelima ini, dikembangkan sistem pengenalan gestur tangan secara real-time dengan memanfaatkan Support Vector Machine (SVM) dan MediaPipe yang terintegrasi dengan OpenCV. Proses diawali dengan memuat dataset keypoint tangan menggunakan fungsi `pd.read_csv()`, kemudian merapikan nama kolom agar seragam. Dataset diasumsikan memiliki label pada kolom pertama dan terdapat noise pada kolom terakhir, sehingga fitur (X) diambil dari kolom kedua hingga kolom sebelum terakhir, sedangkan label (y) berasal dari kolom pertama. Data kemudian dibagi menjadi 80% untuk pelatihan dan 20% untuk pengujian menggunakan `train_test_split()`.

Model SVM dengan kernel linear dilatih menggunakan `SVC(kernel='linear')` dan diuji

pada data pengujian untuk mengukur akurasi menggunakan `accuracy_score()`. Setelah proses pelatihan selesai, model digunakan untuk mendeteksi gestur tangan secara real-time dengan bantuan MediaPipe dan OpenCV. Kamera mengambil citra dan mengonversinya ke format RGB, lalu landmark tangan dideteksi melalui `hands.process()`. Jika tangan terdeteksi, koordinat x, y, dan z dari setiap titik diekstraksi dan diproses oleh model SVM untuk memprediksi gestur tangan tersebut. Hasil prediksi kemudian ditampilkan di layar menggunakan `cv2.putText()`.

Dengan mengkombinasikan MediaPipe sebagai alat pelacak titik kunci tangan dan SVM untuk klasifikasi gestur, sistem ini mampu mengenali posisi dan gerakan tangan secara akurat dalam waktu nyata, memperlihatkan implementasi efektif dari machine learning dan computer vision dalam analisis gerakan manusia.

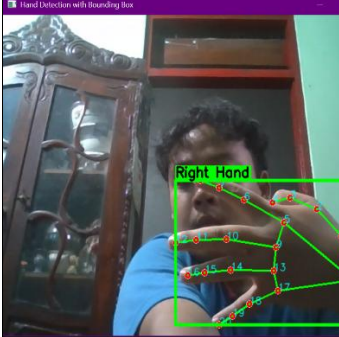
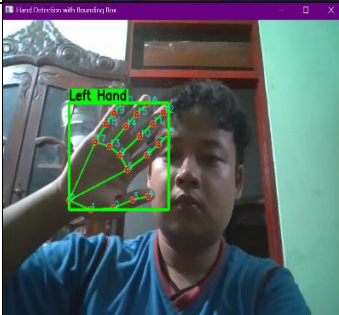
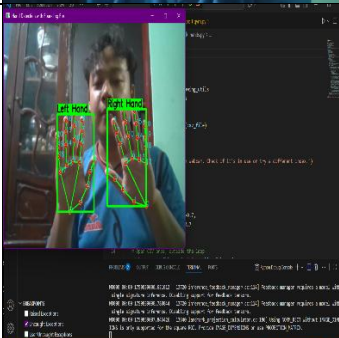
Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini :

No	Bagian Kode / Fungsi	Penjelasan
1	<code>import cv2</code>	Mengimpor pustaka OpenCV untuk menangani pengolahan citra dan akses kamera.
2	<code>import mediapipe as mp</code>	Mengimpor pustaka MediaPipe untuk melakukan deteksi dan pelacakan tangan secara real-time.
3	<code>mp_hands = mp.solutions.hands</code>	Menginisialisasi modul Hands dari MediaPipe yang berfungsi mendeteksi dan melacak titik tangan (landmarks).
4	<code>mp_drawing = mp.solutions.drawing_utils</code>	Menginisialisasi fungsi utilitas untuk menggambar hasil deteksi landmark di atas frame video.
5	<code>cap = cv2.VideoCapture(0)</code>	Membuka koneksi ke kamera (0 berarti kamera bawaan laptop). Jika ada kamera eksternal, indeks bisa diganti (misal 1 atau 2).
6	<code>with mp_hands.Hands(...) as hands:</code>	Membuat objek deteksi tangan dengan parameter: - <code>max_num_hands=2</code> : mendeteksi maksimal dua tangan. - <code>min_detection_confidence=0.7</code> : batas minimal kepercayaan
		deteksi. - <code>min_tracking_confidence=0.7</code> : batas minimal kepercayaan pelacakan.

7	<code>ret, frame = cap.read()</code>	Membaca satu frame dari kamera. <code>ret</code> bernilai <i>True</i> jika pembacaan berhasil.
8	<code>cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)</code>	Mengubah format warna dari BGR (OpenCV) ke RGB (MediaPipe membutuhkan format RGB).
9	<code>results = hands.process(rgb_frame)</code>	Memproses frame RGB untuk mendeteksi tangan dan menghasilkan koordinat titik-titik (landmarks).
10	<code>if results.multi_hand_landmarks:</code>	Mengecek apakah ada tangan yang terdeteksi pada frame.
11	<code>for hand_landmarks in results.multi_hand_landmarks:</code>	Melakukan iterasi untuk setiap tangan yang terdeteksi (bisa satu atau dua).
12	<code>mp_drawing.draw_landmarks(...)</code>	Menggambar titik-titik sendi dan koneksi antar sendi tangan di atas frame video dengan warna dan ketebalan tertentu.
13	<code>cv2.imshow("Deteksi Gerakan Tangan - Titik Sendi", frame)</code>	Menampilkan hasil deteksi tangan secara real-time di jendela OpenCV.
14	<code>if cv2.waitKey(1) & 0xFF == ord('q'):</code>	Memberi opsi keluar dari program jika tombol 'q' ditekan di keyboard.
15	<code>cap.release()</code>	Menutup koneksi kamera setelah program berhenti.
16	<code>cv2.destroyAllWindows()</code>	Menutup semua jendela tampilan OpenCV.

5. Data dan Output Hasil Pengamatan

Tabel berikut menunjukkan hasil pengamatan deteksi gerakan tangan menggunakan sistem berbasis MediaPipe. Setiap gerakan dianalisis berdasarkan jumlah tangan yang terdeteksi, jumlah titik sendi per tangan, posisi atau titik penting pada tangan, serta keterangan visualisasi output yang dihasilkan.

No	Dokumentasi	Deteksi	Jumlah Titik Sendi per Tangan	Posisi/Titik Penting	Keterangan Output Visual
1		Gerakan Tangan kanan membentuk isyarat angka 4 Jumlah Tangan Terdeteksi 1	20 titik	empat jari yang terbuka dan satu jari yang tertutup	Garis hijau menghubungkan sendi, titik merah di setiap sendi
2		Gerakan Tangan kiri membentuk "hai" Jumlah Tangan Terdeteksi 1	21 titik	Tangan Kiri dalam posisi setengah terbuka	Garis berwarna hijau mengikuti kontur jari, sehingga visualisasi sendi tetap presisi.
3		Gerakan Tangan kiri & kanan terbuka lebar Jumlah Tangan Terdeteksi 2	21 titik per tangan	Semua jari terbuka, posisi natural	Garis hijau menghubungkan sendi, titik merah di setiap sendi

6. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, dapat disimpulkan bahwa program yang dikembangkan berhasil melakukan klasifikasi gestur tangan secara real-time dengan menggabungkan model SVM dan MediaPipe. Tingkat akurasi dari sistem ini sangat bergantung pada kualitas dataset serta parameter yang diterapkan; oleh karena itu, peningkatan hasil deteksi dapat dicapai melalui perbaikan dataset, penyesuaian parameter model, dan peningkatan performa dalam proses pengolahan waktu nyata. Sistem ini mampu mengenali gerakan tangan dengan cukup baik, meskipun menghadapi tantangan terkait noise dan kondisi pencahayaan yang kurang optimal. Perlu dicatat bahwa sistem saat ini hanya mampu mendeteksi keberadaan tangan dan maksimal dua tangan sekaligus, sehingga tidak dapat mengenali objek lain maupun lebih dari dua tangan secara bersamaan.

7. Saran

Berikut adalah parafrase dari teks tersebut:

Berdasarkan hasil praktikum dan analisis, terdapat beberapa rekomendasi perbaikan yang dapat dilakukan untuk meningkatkan akurasi serta performa sistem.

1. Peningkatan Dataset: Menambah jumlah dan keragaman sampel agar mencakup berbagai kondisi pencahayaan dan posisi tangan, sehingga model menjadi lebih umum dan akurat.
2. Optimasi Model dan Performa Real-Time: Melakukan penyesuaian hyperparameter pada SVM (seperti parameter C dan gamma), mengoptimalkan kode, melakukan kompresi gambar, serta menerapkan post-processing seperti time smoothing agar prediksi menjadi lebih stabil.
3. Peningkatan Kapasitas Deteksi: Memperluas kemampuan sistem agar bisa mendeteksi lebih dari dua tangan atau objek relevan lainnya secara bersamaan.

10. Daftar Pustaka

Mnih, V., et al. (2015). *Human-level control through deep reinforcement learning*. *Nature*, 518(7540), 529–533.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

AR Ardiansyah. "Implementasi Deteksi Bahasa Isyarat Tangan Menggunakan OpenCV dan MediaPipe," *Jurnal Ilmiah Antivirus*, 2024, menjelaskan penerapan MediaPipe dan SVM untuk klasifikasi bahasa isyarat tangan secara real-time dengan metrik F1 score, recall, dan precision yang tinggi

Y Meng et al. "Model Pemantauan Gerakan Tangan Real-Time Berbasis MediaPipe," *MDPI Sensors*, 2024, mengulas algoritma klasifikasi termasuk SVM dalam sistem monitoring gestur tangan real-time berbasis MediaPipe

R Sreemathy et al., "Pengenalan Bahasa Isyarat Tingkat Kata Berkelanjutan Menggunakan YOLOv4 dan MediaPipe," *ScienceDirect*, 2023, membandingkan metode deteksi dan klasifikasi termasuk SVM dengan hasil akurasi yang tinggi